Juan Estanislao Hoszowski

Backend Software Engineer

+54 11 3852-0980 | juanhoszowski@gmail.com | linkedin.com/in/juanhosz | github.com/juanhosz | Buenos Aires, Argentina

Summary

Backend engineer (Ingeniería en Informática, UBA, Dec 2025) building distributed systems and APIs in Python, TypeScript, C++, and Java. Built crash recovery for a 40-container RabbitMQ pipeline, a 21-endpoint Neo4j microservice end to end, and the networking layer of a multiplayer C++ game server. Each was a team project with a fixed deadline, delivered in tools adopted mid-project (RabbitMQ, Neo4j, Firebase, Qt).

Skills

Languages: Python, TypeScript, C++, Java, Go, SQL
Backend and Testing: Express.js, REST API design, JWT, Firebase Auth, Jest, Supertest, GoogleTest
Data and Messaging: PostgreSQL, Neo4j (Cypher), SQLite, RabbitMQ
Infrastructure: Docker, Docker Compose, GitHub Actions, Linux, Git, CMake, Render, New Relic
Concepts: Distributed Systems, Fault Tolerance, Microservices, Sharding, Multithreading, TCP/IP, Agile/Scrum
Spoken Languages: Spanish (Native), English (Advanced), French (Advanced)

Projects

Fault-Tolerant Distributed Data Pipeline (Team of 3) | **GitHub** Apr 2025 - Jun 2025

Stack: *Python, RabbitMQ, Docker, Docker Compose, TextBlob*

- Built a 40-container pipeline that answers five analytical queries over a 45,000-movie dataset for concurrent clients, with hash-sharded joins and per-stage worker counts generated from a config file.
- Implemented the health-check layer: supervisor processes arranged in a ring probe each other and a disjoint set of nodes over TCP every second, restarting failed containers through Docker-in-Docker.
- Made processing duplicate-free across crashes by switching to manual acks, persisting node state before acknowledging, and discarding replayed messages by sequence number and origin.
- Validated recovery with a chaos script that kills random nodes at a set probability; scaling from one to three workers per stage halved end-to-end time (68 to 34 seconds) for three concurrent clients.

TwitSnap, Microservices Social Platform (Team of 5) | **GitHub** Sep 2024 - Dec 2024

Stack: *TypeScript, Express.js, Neo4j, Jest, Firebase Auth, Java (Spring Cloud Gateway), New Relic, Docker, Render*

- Sole developer of the Twits microservice: 21 REST endpoints (posts, likes, reposts, comments, feed, trending, search, moderation) in a layered controller-service-repository design with dependency injection.
- Modeled posts, interactions, and hashtags as a Neo4j graph and wrote the Cypher queries behind the feed, interest-based account recommendations, and trending topics, applying privacy and block rules in every query.
- Implemented Google sign-in (Firebase Admin) and JWT validation in the Auth service, added New Relic metrics to the Java API gateway, and covered both Node services with Jest and Supertest integration tests.

Multiplayer Artillery Game, Worms-style (Team of 4, 7 weeks) | **GitHub** Oct 2023 - Dec 2023

Stack: *C++, TCP sockets, multithreading, Box2D, SDL2, Qt, CMake, GoogleTest, YAML*

- Owned client-server networking: a binary TCP protocol with 23 opcodes, a sender and a receiver thread per client over blocking queues, and Command-pattern dispatch of player input into the game loop.
- Built the server's match lifecycle: a lobby running concurrent matches, a 30 Hz fixed-timestep loop per match, a turn-manager state machine (turn timer, bonus time, wait for physics to settle), and cleanup of finished matches.
- Sent static map data once in a start-of-match handshake instead of in every snapshot; fixed server memory leaks by moving ownership to smart pointers; added GoogleTest coverage for the socket layer.
- Team shipped 10 weapons, 18 maps, and a map editor while adopting Box2D, SDL2, and Qt mid-build.

Education

Universidad de Buenos Aires (UBA), Facultad de Ingeniería Buenos Aires, Argentina

Ingeniería en Informática (Computer Engineering) - Graduated *Mar 2018 - Dec 2025*